

RETIMING RAK Using Genus

cadence®

What is Retiming ?

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

CONCEPT

- Technique for improving the performance of sequential circuits by repositioning registers to reduce the cycle time or the area without changing the input-output latency

- Why Retiming ?
  - Retiming allows you to improve the performance of the design during synthesis without having to redesign the RTL
  - Retiming adds a new dimension to optimization by allowing registers to move over combinational logic

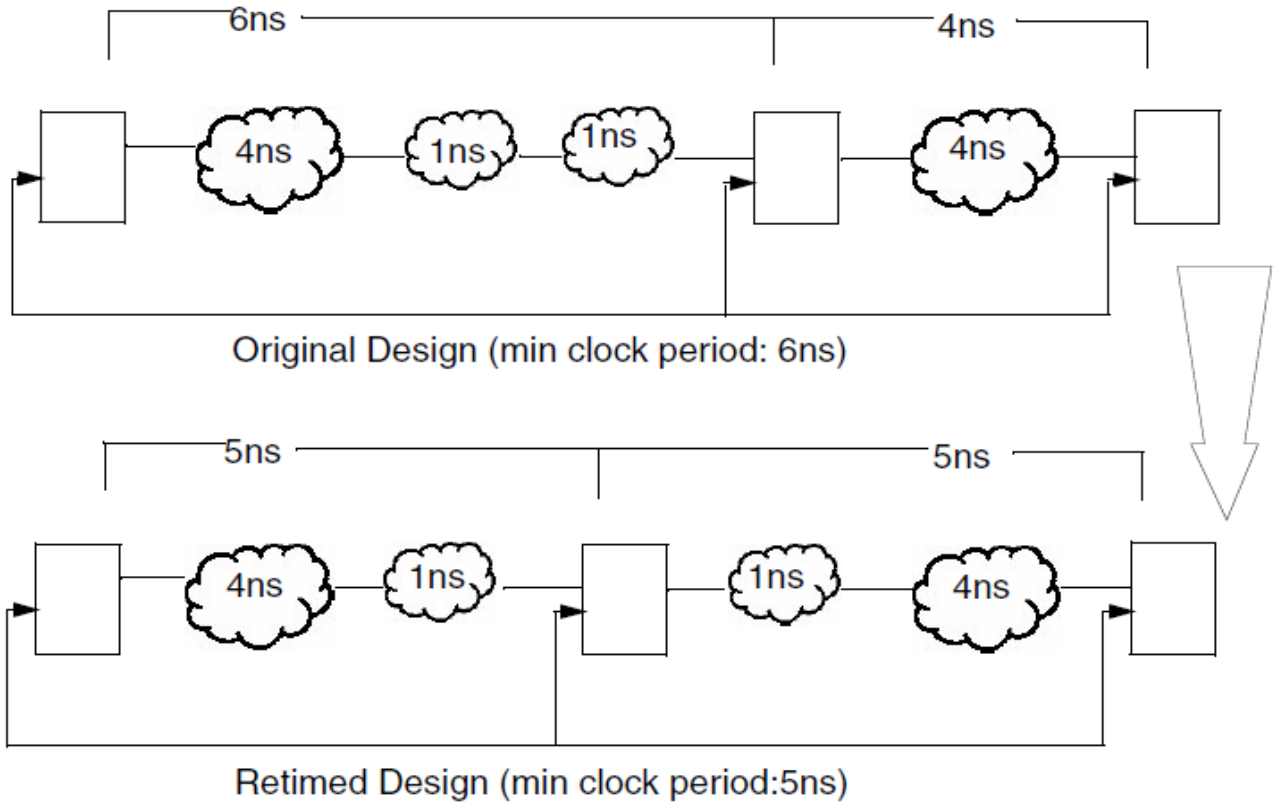
- How is it different than pipelining ?
  - Pipelining is a subset of retiming where sufficient stages are added to the design
  - Retiming operation distributes the sequential elements at appropriate locations to meet performance requirements

- Which metrics would retiming help?
  - Performance
  - Area

Genus supports both automatic and manual retiming

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

RETIMING FOR TIMING

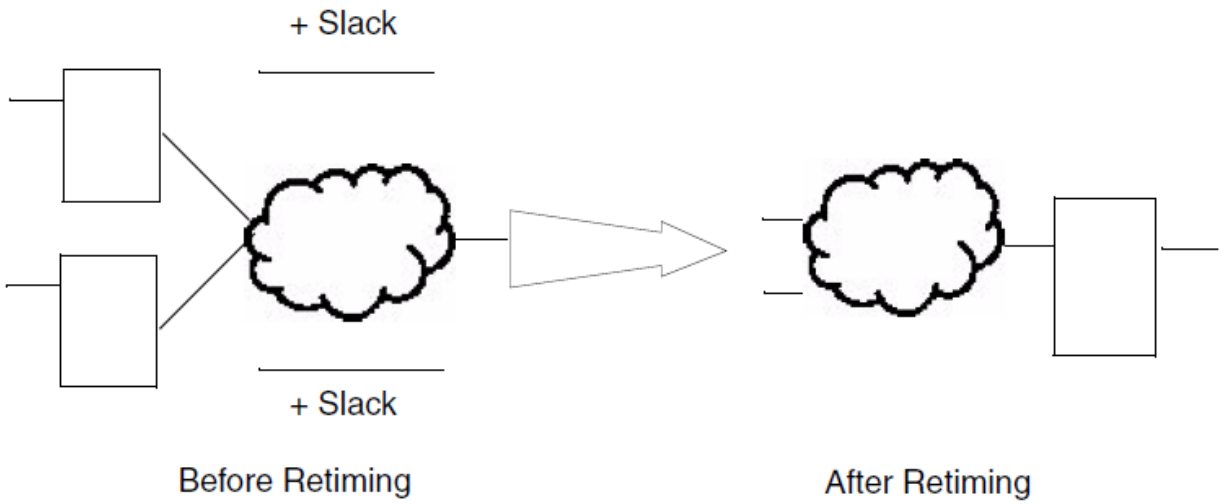


Retiming reduces the clock period from 6ns to 5ns

- Retiming tries to balance the delays on the worst critical paths, thereby reducing the WNS.
- The number of registers in the design before retiming may not be the same after retiming because some of the registers may have been combined or replicated.

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

RETIMING FOR AREA



Retiming reduces the register count from 2 to 1

- When retiming for area, Genus moves registers in order to minimize the register count without worsening the critical path in the design.

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

AUTOMATIC TOP-DOWN RETIMING FLOW

set_attribute retime true {submod1 submod2} – will retime sub modules submod1 and submod2

set_attribute retime true {design} – will retime the entire design

- In this flow retiming focuses on minimizing the delay.
- Use this flow when retiming is part of a well-planned synthesis strategy and the design has sub designs that can be retimed.
- Genus automatically derives appropriate constraints, synthesizes, and retimes the specified modules.

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

AUTOMATIC TOP-DOWN RETIMING FLOW

```
read_hdl top.v
elaborate top
set_attribute retime true /design/top        //retime the entire design
read_sdc top.sdc
syn_gen
syn_map
syn_opt
report timing
```

Retiming is performed automatically on the blocks marked with retime attribute during `syn_gen` (generic synthesis stage)

By default automatic retiming is used.

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

MANUAL RETIMING (BLOCK_LEVEL RETIMING)

- Manual Retiming is used for debugging purpose. For example if a particular sub-block in the design is not getting retimed, manual retiming can be used to debug.
- Manual retiming does not involve a flow, like automatic top-down retiming.

- Retiming for Minimum Delay –
  - Perform block level retiming on a block by block basis to further optimize the design, thereby minimizing the delay or area. Manual flow should be done on a post elaboration netlist.

```
retime –prepare
retime –min_delay
```

- -`prepare` : It prepares the design for retiming by deriving the appropriate constraints and synthesizing to a gate-level design that is ready for retiming.
- When retiming is subsequently performed, the original constraints will be used and not those derived with the -`prepare` option.

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

Retiming Analysis

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

AUTOMATIC TOP-DOWN RETIMING FLOW

- keywords : Pre-retime summary/ Post-retime summary

  Present in super_thread_debug directories

  Since this RAK design is small, retiming information is present in the main log, but practically for a large design (instance count > 100K) the retiming info would be present in super_thread_debug directories

Indicates the beginning of retiming

WNS before retiming

```
Pre-retime summary
==========================
Slack               : -1450 ps
Number of registers : 250

Retiming testdesign
  Preparing testdesign for retiming.
  Analyzing design.
  Decomposing flops
  Analyzing retimeable flops
Info    : Categorized flops into classes. [RETIME-501]
        : Created 2 flop classes for 250 flops.

            class 1 contains 60 flops.
              clock source           (pos) /designs/testdesign/ports_in/clk
            class 2 contains 190 flops.
              clock source           (pos) /designs/testdesign/ports_in/clk
              asynchronous reset/set (neg) /designs/testdesign/ports_in/rst_l

        : Only flops in the same class can merge during a retiming move.
        Tracing clock networks.
        Levelizing the circuit.
        Applying wireload models.
        Computing net loads.
        Computing delays.
        Computing arrivals and requireds.
    Creating boundaries.
    Solving retiming problem.
Info    : Retimed asynchronous reset behavior could not be preserved without negatively impacting timing. [RETIME-113]
        : Inserting gates for explicit reset and redoing retiming.  Disable with retime_fallback_to_explicit_reset root-level attribute.
    Creating boundaries.
    Solving retiming problem.
    Implementing retiming solution.

Post-retime summary
==========================
Slack               : -944 ps
Number of registers : 737
```

Improved WNS

Indicates the completion of retiming

WNS after retiming

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

Debugging

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

AUTOMATIC TOP-DOWN RETIMING FLOW

- What happens to the critical path ?
  Critical path usually changes upon retiming.
- The tool estimates the critical path before retiming.
- Now, retiming will try to balance the delays on all the paths such that the delay does not exceed the WNS estimated.
- After balancing the delays, it so might happen that the huge WNS estimated gets distributed along other paths and we get a new critical path with different (reduced) WNS.
- Following attributes used for debugging the critical path
  - *set_attr retime_debug 1 /*
  - *set_attr retime_debug_path 1 /*
- Estimated Critical path

```
686 retime: The critical path in the design is:
687 retime:   register/clk (xout_15_4_pre_reg[6]) (class 1)          0.00         0.00
688 retime:   register/q (xout_15_4_pre_reg[6]) (class 1)          224.30       224.30
689 retime:   Port: csa_tree_add_94_52_group_4887/in_0[6]/in_0            ( 0)         0.00       224.30
690 retime:   Port: csa_tree_add_94_52_group_4887/in_0[6]/in_0            ( 0)         0.00       224.30
691 retime:   csa_tree_add_94_52_groupi/g18/in_0             ( 0)         0.00       224.30
```

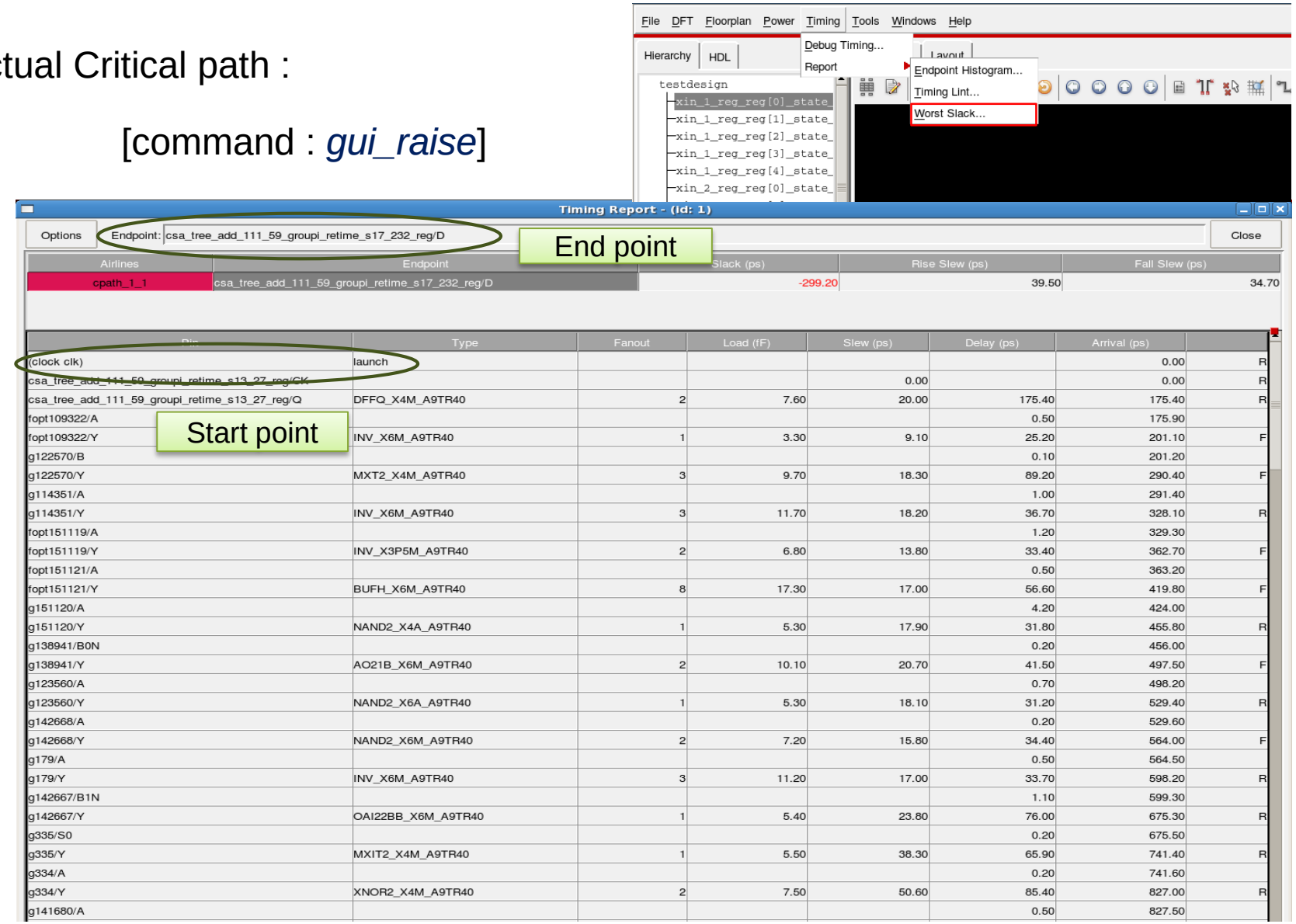
```
864 retime:   csa_tree_add_145_59_groupi/g2539/z             ( 0)        80.50      5748.30
865 retime:   Pin: /csa_tree_add_145_59_groupi/out_0[12]/out_0             ( 0)         0.00      5748.30
866 retime:   Pin: /csa_tree_add_145_59_groupi/out_0[12]/out_0             ( 0)         0.00      5748.30
867 retime:   register/d (xout_15_4_pre_reg[6]) (class 1)           0.00      5748.30
868 retime:   register/clk (xout_15_4_pre_reg[6]) (class 1)         52.20      5800.50
```


Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

USING GUI

- Actual Critical path :

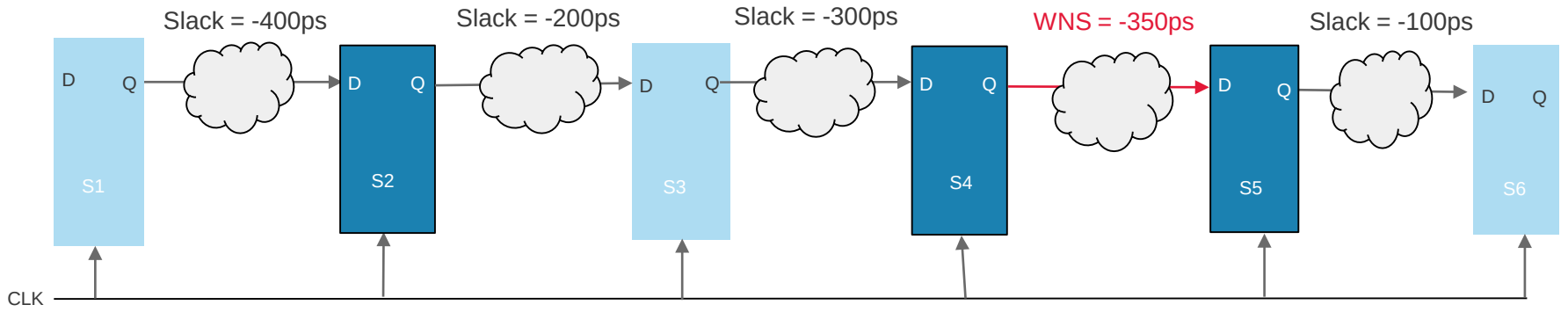
[command : *gui_raise*]



Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

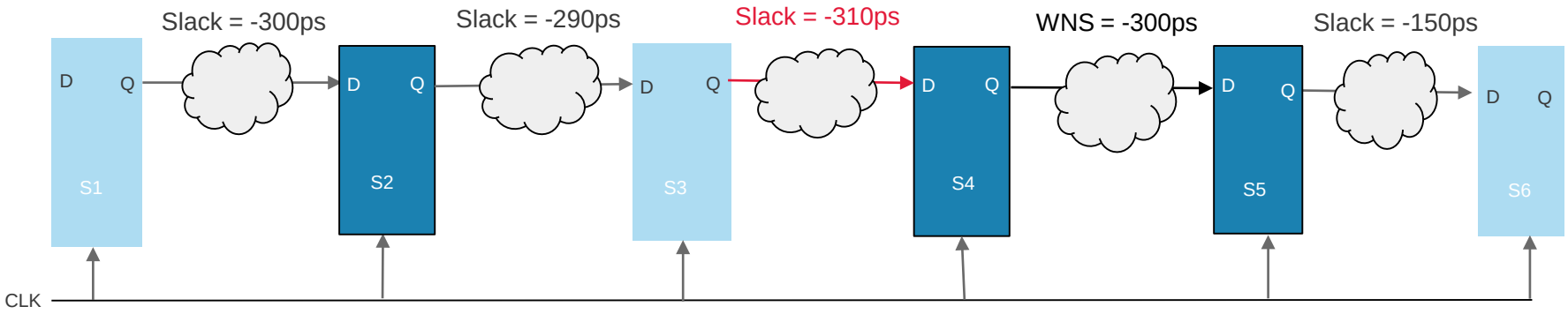
AUTOMATIC TOP-DOWN RETIMING FLOW

- Consider a simple example
  Before Retiming



- After Retiming

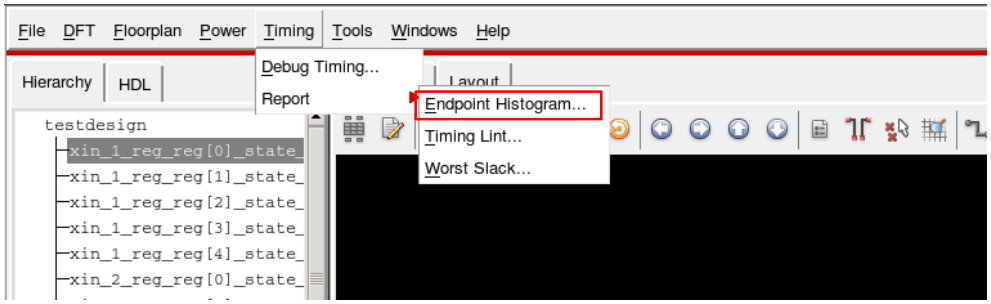
Critical path changed from S4-S5 (WNS -350ps) to S3-S4(WNS -310ps)

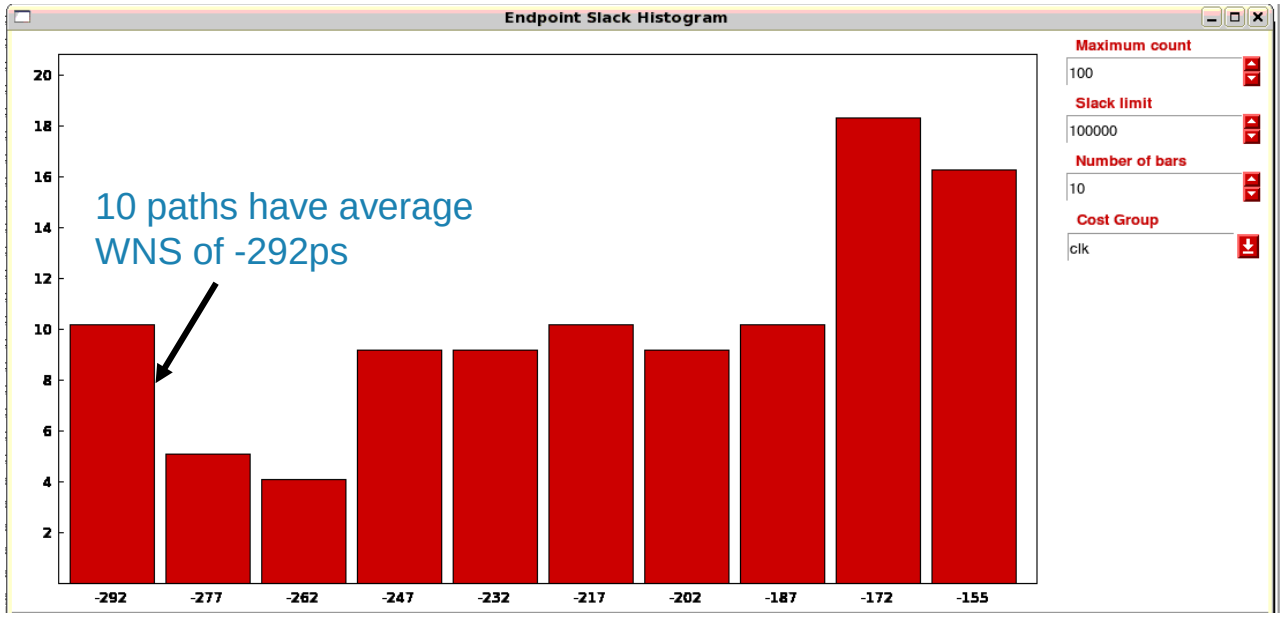


Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

AUTOMATIC TOP-DOWN RETIMING FLOW

- Raise GUI and report timing histogram to see the range of average WNS values





Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.



Lab

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

Steps to Follow

- Download and install the Genus 16.1/16.2 software from https://downloads.cadence.com.

- Testcase database, Scripts and references can be found at ‘Attachments’ and ‘Related Solutions’ sections below the PDF. This pdf can be searched with the document 'Title – Retiming RAK Using Genus' on https://support.cadence.com

- Test design is - Retiming_RAK.tar.gz

  This test case is an example for automatic retiming flow

- Extract RAK database –

❖ *tar -xvfz Retiming_RAK.tar.gz*

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.

Steps to Follow

- Open Genus –
  - *setup Genus 16.20 (2016)*
  - *genus –legacy_ui –f run.tcl –log RAK_log.log*
- *User can change the tcl script and update the log file name accordingly.*

- To open GUI –
  - after the run is complete

- To enable retiming set the attribute in the script as follows :
  - *set_attribute retime true*

- To disable retiming set the attribute in the script as follows :
  - *set_attribute retime false*

Copyright © 2017 Cadence Design Systems, Inc. All rights Reserved.


cadence®